MAESTRÍA EN CIENCIA DE DATOS

Práctica 7

Algoritmos de Clasificación en R

LDA, CART, kNN, SVM lineal y Random Forest sobre niveles de obesidad en personas de México, Perú y Colombia (Kaggle), comparados con **caret**

Elaborado por: Miguel Deza

Programa: Maestría en Ciencia de Datos

Curso: Introducción a Ciencia de Datos

Docente: Yessenia Deysi Yari Ramos

Fecha: 21 de septiembre de 2026

1 Desarrollo

1.1 1. Cargar las bibliotecas necesarias

```
library(caret)
library(MASS)           # LDA
library(rpart)           # CART
library(rpart.plot)
library(e1071)            # SVM (y soporte interno de caret)
library(kernlab)           # motor de SVM lineal usado por caret
library(randomForest)
library(corrplot)
library(ggplot2)
```

Todo cargó bien.

1.2 2. Cargar el conjunto de datos (CSV)

Busqué en https://www.kaggle.com/datasets?tags=13302-Classification un dataset de mi interés e inicié sesión en Kaggle para descargarlo directamente. Elegí **Obesity Levels** (`fatemehmehrparvar/obesity-levels`): estimación del nivel de obesidad de personas de **México, Perú y Colombia** según hábitos alimenticios y actividad física.

```
datos <- read.csv("obesidad.csv", stringsAsFactors = TRUE)
niveles_obesidad <- c("Insufficient_Weight", "Normal_Weight",
                      "Overweight_Level_I", "Overweight_Level_II",
                      "Obesity_Type_I", "Obesity_Type_II", "Obesity_
                         Type_III")
datos$NObeyesdad <- factor(datos$NObeyesdad, levels = niveles_obesidad)
str(datos)
```

```
'data.frame': 2111 obs. of  17 variables:
 $ Age                           : num  21 21 23 27 22 29 23 22 24 22 ...
 $ Gender                        : Factor w/ 2 levels "Female","Male"
 $ Height                        : num  1.62 1.52 1.8 1.8 1.78 ...
 $ Weight                        : num  64 56 77 87 89.8 53 55 53 64 68 ...
 $ CALC                          : Factor w/ 4 levels "Always","Frequently",...
 $ FAVC                          : Factor w/ 2 levels "no","yes"
 $ FCVC                          : num  2 3 2 3 2 2 3 2 3 2 ...
 $ NCP                           : num  3 3 3 3 1 3 3 3 3 3 ...
 $ SCC                           : Factor w/ 2 levels "no","yes"
 $ SMOKE                         : Factor w/ 2 levels "no","yes"
 $ CH2O                          : num  2 3 2 2 2 2 2 2 2 2 ...
 $ family_history_with_overweight: Factor w/ 2 levels "no","yes"
 $ FAF                           : num  0 3 2 2 0 0 1 3 1 1 ...
 $ TUE                           : num  1 0 1 0 0 0 0 0 1 1 ...
 $ CAEC                          : Factor w/ 4 levels "Always","Frequently",...
 $ MTRANS                        : Factor w/ 5 levels "Automobile","Bike",...
 $ NObeyesdad                    : Factor w/ 7 levels "Insufficient_Weight",...
```

2111 personas, 16 predictores (edad, género, altura, peso, hábitos de alimentación/actividad física y transporte) y la clase `NObeyesdad` con 7 niveles de obesidad.

1.3 3. Análisis exploratorio de datos

```
summary(datos)
table(datos$NObeyesdad)
colSums(is.na(datos))
cor(datos[, c("Age","Height","Weight","FCVC","NCP","CH2O","FAF","TUE")
    ])
```

```
Balance de clases:
Insufficient_Weight       Normal_Weight  Overweight_Level_I Overweight_Level_II
                272                 287                 290                290
     Obesity_Type_I     Obesity_Type_II    Obesity_Type_III
                351                 297                 324

Proporcion (%): 12.9  13.6  13.7  13.7  16.6  14.1  15.3

Valores faltantes: 0 en las 17 columnas
```

Clase **bien balanceada** (12.9 %–16.6 % por nivel) y **sin valores faltantes**. En la correlación entre numéricas, `Height–Weight` es la más fuerte (0.46), seguida de `Height–FAF` (0.29) y `Age–TUE`, la única negativa relevante (-0.30: las personas mayores usan menos dispositivos electrónicos).

1.4 4. Visualizar el conjunto de datos

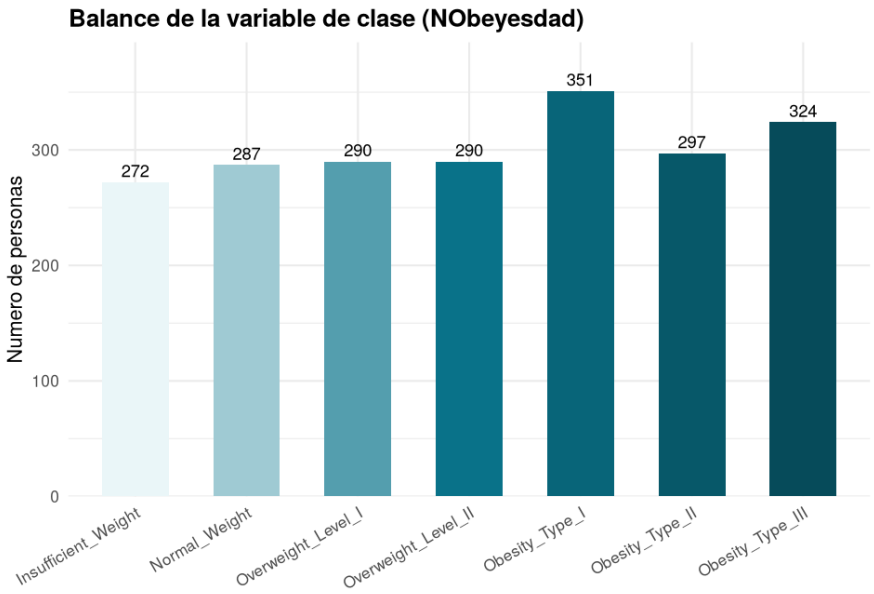


Figura 1: *Balance de clases. Uso una paleta secuencial (no categórica) porque `NObeyesdad` es ordinal: a mayor nivel, más obesidad.*

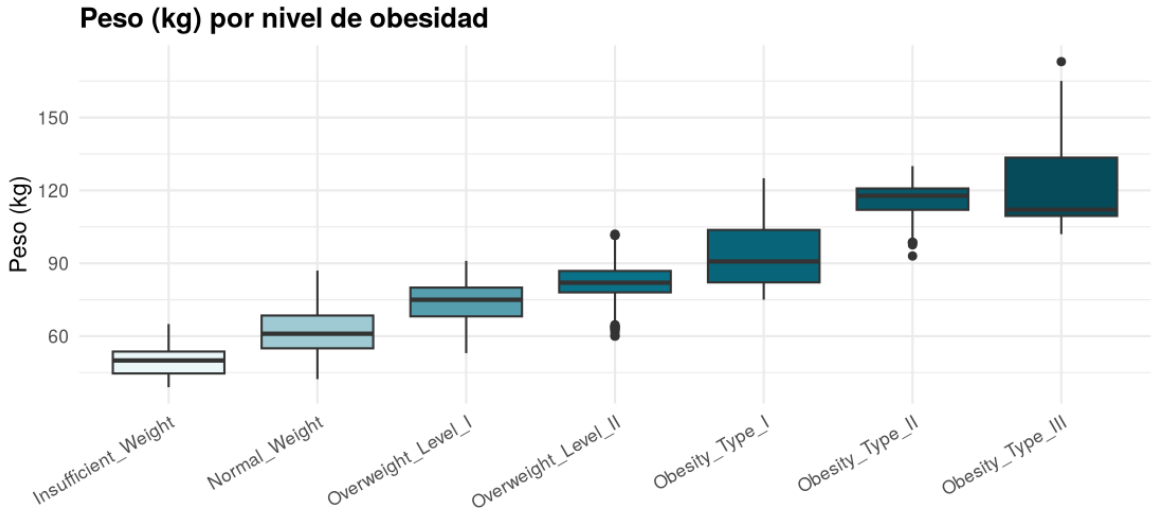


Figura 2: *Peso por nivel de obesidad: la relación es perfectamente monótona, tal como debe ser por construcción de las categorías.*



Figura 3: *Correlación entre los 8 predictores numéricos.*

1.5 5. Preprocesar los datos

```
set.seed(7)
idx_train <- createDataPartition(datos$NObeyesdad, p = 0.80, list =
   FALSE)
train <- datos[idx_train, ]
test  <- datos[-idx_train, ]
```

```
Filas train / test:
train  test
 1691   420
```

No hizo falta imputar nada (sin NA). Las 8 columnas categóricas (`Gender`, `CALC`, `FAVC`, `SCC`, `SMOKE`, `family_history_with_overweight`, `CAEC`, `MTRANS`) ya quedaron como `factor`; `train()` las convierte a variables *dummy* automáticamente vía la fórmula. Centré y escalé las numéricas dentro de cada `train()` (`preProcess = c("center","scale")`) porque kNN y SVM son sensibles a la escala.

1.6 ▶ 6. Ejecutar los algoritmos

```
control <- trainControl(method = "repeatedcv", number = 10, repeats =
    3)
metric  <- "Accuracy"
prep    <- c("center", "scale")

fit.lda  <- train(NObeyesdad ~ ., data = train, method = "lda",
                  metric = metric, trControl = control, preProcess =
                     prep)
fit.cart <- train(NObeyesdad ~ ., data = train, method = "rpart",
                  metric = metric, trControl = control, tuneLength =
                     8)
fit.knn  <- train(NObeyesdad ~ ., data = train, method = "knn",
                  metric = metric, trControl = control, preProcess =
                     prep, tuneLength = 10)
fit.svm  <- train(NObeyesdad ~ ., data = train, method = "svmLinear",
                  metric = metric, trControl = control, preProcess =
                     prep)
fit.rf   <- train(NObeyesdad ~ ., data = train, method = "rf",
                  metric = metric, trControl = control)
```

Los cinco modelos usan la misma validación cruzada: **10 pliegues** × **3 repeticiones**. Al entrenar LDA, R avisó `"variables are collinear"` – con 8 columnas categóricas convertidas a *dummies* (algunas con niveles muy raros, como `CALC="Always"` con solo 1 caso), el sistema de ecuaciones de LDA queda casi singular; no impide el ajuste, pero es una limitación real de LDA frente a datos con muchas variables *dummy* poco frecuentes.

1.7 📊 7. Evaluar los algoritmos

```
resultados <- resamples(list(LDA=fit.lda, CART=fit.cart, kNN=fit.knn,
                             SVM=fit.svm, RF=fit.rf))
summary(resultados)
```

```
Accuracy (media de 30 evaluaciones)
LDA  0.8965   CART 0.7765   kNN 0.7905   SVM 0.9515   RF 0.9645
Kappa (media)
LDA  0.8791   CART 0.7392   kNN 0.7553   SVM 0.9433   RF 0.9585
```

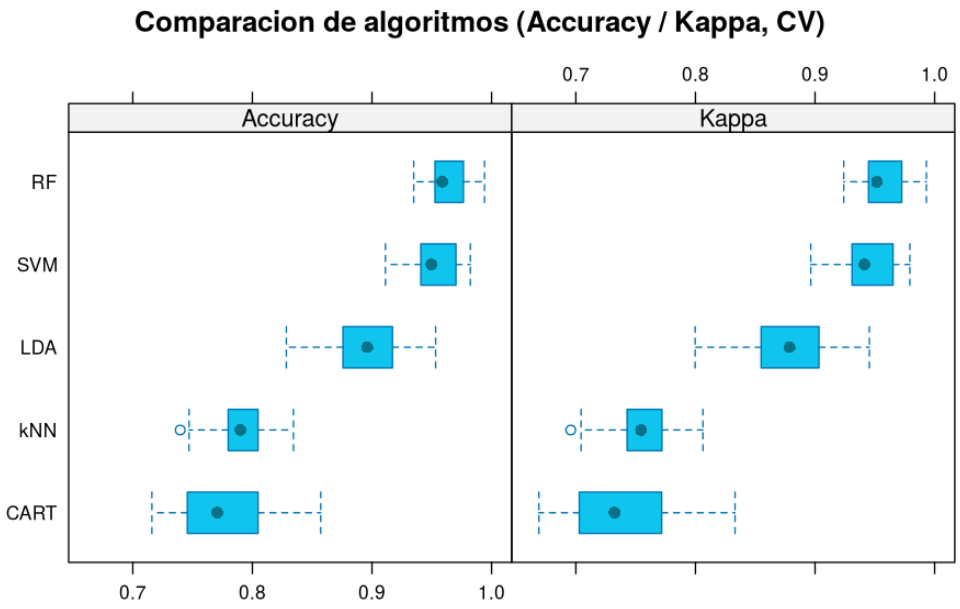


Figura 4: *Accuracy y Kappa de las 30 evaluaciones de validación cruzada, por algoritmo. Jerarquía clara: RF > SVM > LDA > kNN > CART.*

```
pred.lda <- predict(fit.lda, test); pred.rf <- predict(fit.rf, test)
confusionMatrix(pred.lda, test$NObeyesdad)
# ... (una matriz de confusion 7x7 por modelo)
```

Modelo	Accuracy CV	Accuracy test	Kappa test	Sensib. macro	Especif. macro
LDA	0.896	0.879	0.858	0.875	0.980
CART	0.777	0.783	0.747	0.789	0.964
kNN	0.790	0.819	0.789	0.815	0.970
SVM lineal	0.951	0.943	0.933	0.941	0.991
Random Forest	**0.965**	**0.969**	**0.964**	**0.969**	**0.995**

Cuadro 1: *Tabla comparativa final sobre el conjunto de prueba (420 personas). Sensibilidad/especificidad son el promedio macro de las 7 clases.*

Random Forest es claramente el mejor (96.9 % de accuracy en prueba, Kappa 0.964), seguido de SVM lineal (94.3 %). CART, el modelo más simple, es el más débil (78.3 %) – señal de que las fronteras entre niveles de obesidad no son un solo corte rectangular, sino que se benefician de un ensamble de árboles o un margen lineal de alta dimensión.

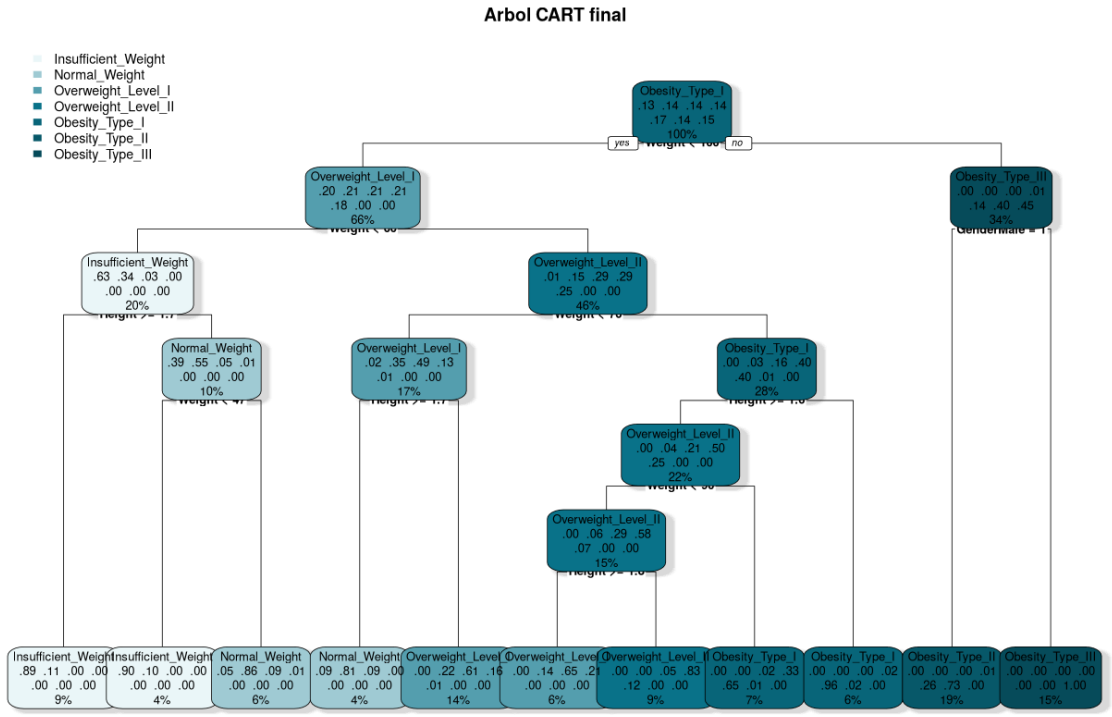


Figura 5: *Árbol CART final: 6 niveles de profundidad, dividiendo primero por* `Weight` *y luego por* `Height` *y* `Gender`.

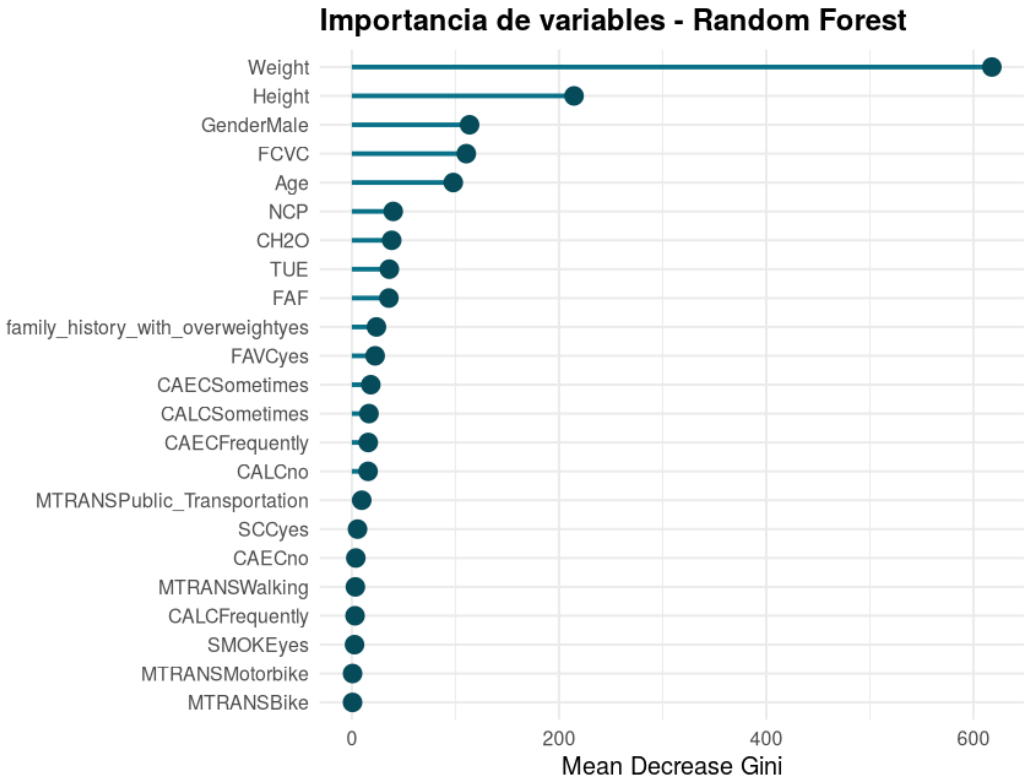


Figura 6: *Importancia de variables (Random Forest, Mean Decrease Gini):* ***`Weight`*** *y* ***`Height`*** *dominan por un margen amplio.*

Tanto el árbol CART como Random Forest coinciden en que `Weight` y `Height` son los predictores más fuertes – razonable, dado que la clase `NObeyesdad` está construida a partir del IMC (peso/altura^2).

1.8 8. Hacer predicciones

Con Random Forest (mejor modelo), sobre una muestra del set de prueba:

```
muestra <- test[sample(nrow(test), 6), ]
predict(fit.rf, muestra)
```

```
                  Real           Prediccion
1      Obesity_Type_I      Obesity_Type_I
2    Obesity_Type_III    Obesity_Type_III
3 Insufficient_Weight Insufficient_Weight
4 Overweight_Level_II Overweight_Level_II
5      Obesity_Type_I      Obesity_Type_I
6 Overweight_Level_II Overweight_Level_II
```

6 de 6 aciertos.

Y sobre una persona nueva (hipotética, perfil peruano típico):

```
persona_nueva <- data.frame(
  Age = 27, Gender = "Male", Height = 1.72, Weight = 88,
  CALC = "Sometimes", FAVC = "yes", FCVC = 2, NCP = 3, SCC = "no",
     SMOKE = "no",
  CH2O = 2, family_history_with_overweight = "yes", FAF = 1, TUE = 2,
  CAEC = "Sometimes", MTRANS = "Public_Transportation")
predict(fit.rf, persona_nueva)
predict(fit.rf, persona_nueva, type = "prob")
```

```
[1] Overweight_Level_II
  Insufficient_Weight Normal_Weight Overweight_Level_I Overweight_Level_II
1                   0         0.002               0.07                0.854
  Obesity_Type_I Obesity_Type_II Obesity_Type_III
1          0.074               0                0
```

Con 1.72 m y 88 kg (IMC $\approx$ 29.7), el modelo predice `Overweight_Level_II` con 85.4 % de probabilidad – consistente con el rango de IMC de esa categoría.